第10章 内部排序

10.1 概述
10.2 插入排序
10.3 交换排序
10.4 选择排序
10.5 归并排序
10.6 基数排序

10.3 交换排序

交换排序的基本思想是：

两两比较待排序记录的关键码，如果发生逆序（即排列顺序与排序后的次序正好相反），则交换之，直到所有记录都排好序为止。

交换排序的主要算法有：

1) 冒泡排序

2) 快速排序

1) 冒泡排序

基本思路：每趟不断将记录两两比较，并按“前小后大”（或“前大后小”）规则交换。

优点：每趟结束时，不仅能挤出一个最大值到最后面位置，还能同时部分理顺其他元素；一旦下趟没有交换发生，还可以提前结束排序。

前提：顺序存储结构

例：关键字序列 T=(21，25，49，25*，16，08），请写出冒泡排序的具体实现过程。

初态：	21，	25，	49，	25*，	16，	08
第1趟	21，	25，	25*，	16，	08，	49
第2趟	21，	25，	16，	08，	25*，	49
第3趟	21，	16，	08，	25，	25*，	49
第4趟	16，	08，	21，	25，	25*，	49
第5趟	08，	16，	21，	25，	25*，	49

冒泡排序的优点：每一趟整理元素时，不仅可以完全确定一个元素的位置（挤出一个泡到表尾），还可以对前面的元素作一些整理，所以比一般的排序要快。

有没有比冒泡排序更快的算法?

有!

快速排序法——全球公认!

因为它每趟都能准确定位不止1个元素!

2） 快速排序

基本思想：

从待排序列中任取一个元素 (例如取第一个) 作为中心，所有比它小的元素一律前放，所有比它大的元素一律后放，形成左右两个子表；

然后再对各子表重新选择中心元素并依此规则调整，直到每个子表的元素只剩一个。此时便为有序序列了。

优点：**因为每趟可以确定不止一个元素的位置，而且呈指数增加，所以特别快！**

前提：顺序存储结构

例1：关键字序列 T=(21，25，49，25*，16，08），请写出快速排序的算法步骤。

设以首元素为枢轴中心

初态：　21，25，49，25*，16，08

第1趟：（16，08），21，（25，25*，49）

第2趟：　(08)，16，21，25，(25*，49)

第3趟：　08，16，21，25，25*，(49)

讨论：

1. 这种不断划分子表的过程，计算机如何自动实现?

2. “快速排序”是否真的比任何排序算法都快?

例2：关键字序列 T=(21，25，49，25*，16，08），计算机如何实现快速排序算法的某一趟过程？

设计技巧：**交替/振荡式逼近**

low

pivotkey=21

high

r[i]	0	1	2	3	4	5	6
初态		21	25	49	25*	16	08
第1趟	21	08	16	21	25*	49	25

（ 08，16） 21 （ 25*， 49， 25 ）

Low=high=3，本趟停止，将中枢点定位并返回位置信息

25* 跑到了前面，不稳定！

例3：以关键字序列（256，301，751，129，937，863，742，694，076，438）为例，写出执行快速算法的各趟排序结束时，关键字序列的状态。

意即模拟算法实现步骤

原始序列： 256，301，751，129，937，863，742，694，076，438

快速排序

第1趟 076，129，256，751，937，863，742，694，301，438

第2趟 076，129，256，438，301，694，742，751，863，937

第3趟 076，129，256，301，438，694，742，751，863，937

第4趟 076，129，256，301，438，694，742，751，863，937

时间效率： $O(n\log_2 n)$ —因为每趟确定的元素呈指数增加

空间效率： $O(\log_2 n)$ —因为递归要用栈(存每层low，high和pivot)

稳 定 性： 不 稳 定 —因为有跳跃式交换。

讨论1：如何编程实现?

分析：

①每一趟子表的形成是采用从两头向中间交替式逼近法；

②由于每趟中对各子表的操作都相似，主程序可采用递归算法。

一趟快速排序算法（针对一个子表的操作）

见教材P275

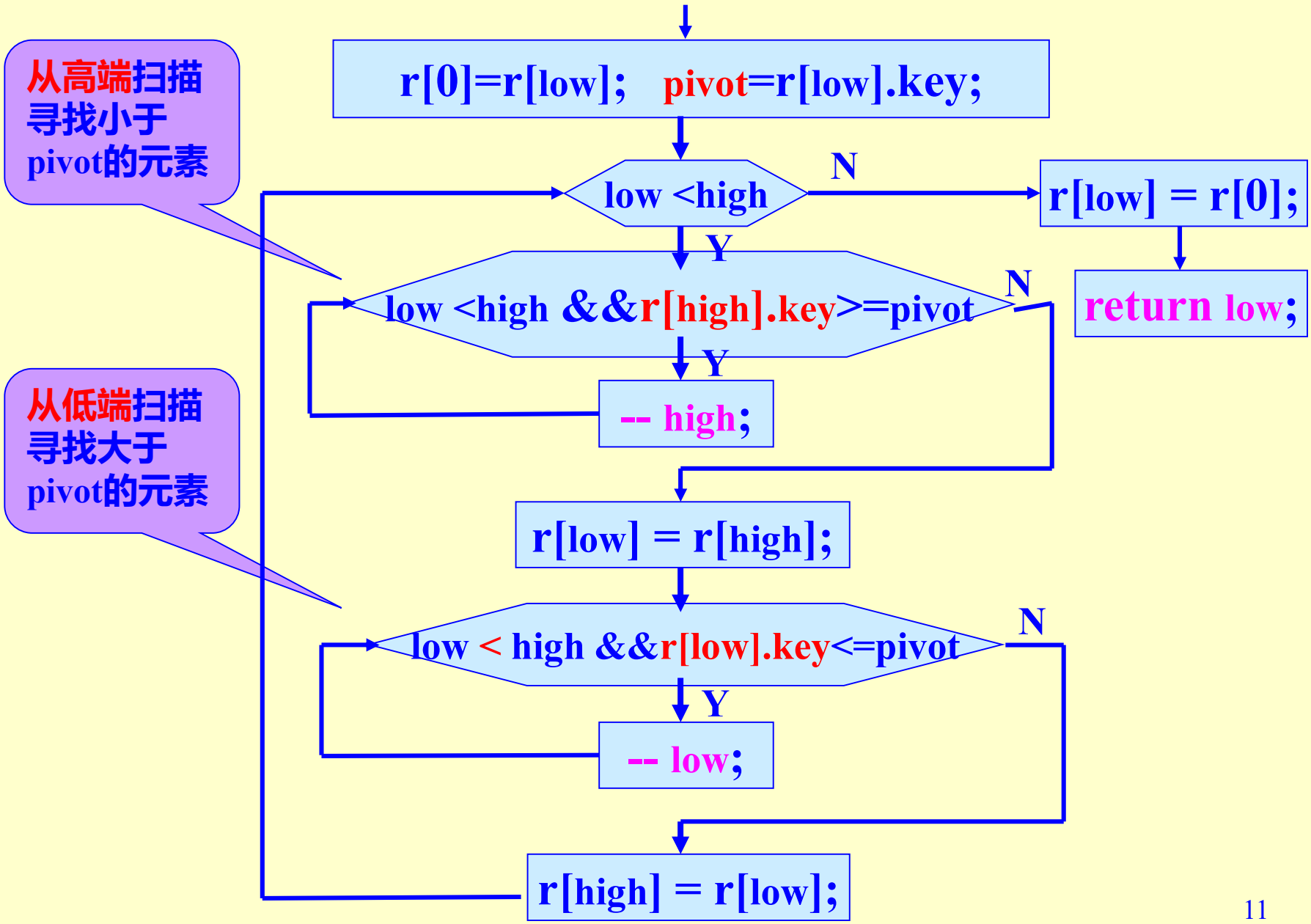
```
int Partition(SqList &L,int low,int high){ //一趟快排
//交换子表 r[low…high]的记录，使支点（枢轴）记录到位，并返回其位置。
返回时，在支点之前的记录均不大于它，支点之后的记录均不小于它。

r[0]=r[low]; //以子表的首记录作为支点记录，放入r[0]单元
```

(续下页)

```
pivotkey=r[low].key;  //取支点的关键码存入pivotkey变量
while(low < high){        //从表的两端交替地向中间扫描
      while(low<high && r[high].key>=pivotkey ) - -high;
        r[low]=r[high];  //比支点小的记录交换到低端;
      while(low<high && r[low].key<=pivotkey)  + +low;
        r[high]=r[low];   //比支点大的记录交换到高端;
      }
   r[low]=r[0];     //支点记录到位;
          return low;    //返回支点记录所在位置。
   }//Partition
```

一趟快速排序算法流程图



整个快速排序的递归算法：

见教材P276

```
void QSort ( SqList  &L,  int low, int high ) {
            //对顺序表L中的子序列r[ low…high] 作快速排序
 if ( low < high) {              //长度>1
     pivot = Partition ( L, low, high ); //一趟快排，将r[ ]一分为二
     QSort ( L, low, pivot-1);    //在左子区间进行递归快排，直到长度为1
     QSort ( L, pivot+1, high ); //在右子区间进行递归快排，直到长度为1
   } //if
} //QSort
```

pivot是局部变量

对顺序表L进行快速排序的操作函数为：

```
void QuickSort ( SqList  &L) {
  QSort ( L,  1,  L.length );
}
```

讨论2. “快速排序”是否真的比任何排序算法都快?

——基本上是，因为每趟可以确定的数据元素是呈指数增加的。

设每个子表的支点都在中间（比较均衡），则：

第1趟比较，可以确定1个元素的位置；
第2趟比较（2个子表），可以再确定2个元素的位置；
第3趟比较（4个子表），可以再确定4个元素的位置；
第4趟比较（8个子表），可以再确定8个元素的位置；
……
只需$\lfloor \log_2 n \rfloor + 1$趟便可排好序。

而且，每趟需要比较和移动的元素也呈指数下降，加上编程时使用了交替逼近技巧，更进一步减少了移动次数，所以速度特别快。

教材P276有证明：快速排序的平均排序效率为$O(n\log_2 n)$；
但最坏情况下**(例如天然有序)**仍为$O(n^2)$，改进措施见P277。

10.4 选择排序

选择排序的基本思想是：每一趟在后面n-i 个待排记录中选取关键字最小的记录作为有序序列中的第i 个记录。

选择排序有多种具体实现算法：

1) 简单选择排序

2) 锦标赛排序

3) 堆排序

1）简单选择排序

思路异常简单：每经过一趟比较就找出一个最小值，与待排序列最前面的位置互换即可。

——首先，在n个记录中选择最小者放到r[1]位置；然后，从剩余的n-1个记录中选择最小者放到r[2]位置；…如此进行下去，直到全部有序为止。

优点：实现简单

缺点：每趟只能确定一个元素，表长为n时需要n-1趟

前提：顺序存储结构

例： 关键字序列T=（21，25，49，25*，16，08），请给出简单选择排序的具体实现过程。

最小值 08 与 r[1]交换位置

原始序列： 21，25，49，25*，16，08

直接选择排序

第1趟	08，25，49，25*，16，21
第2趟	08，16，49，25*，25，21
第3趟	08，16，21，25*，25，49
第4趟	08，16，21，25*，25，49
第5趟	08，16，21，25*，25，49

时间效率： $O(n^2)$——虽移动次数较少，但比较次数仍多。

空间效率： O（1）——没有附加单元（仅用到1个temp）

算法的稳定性： 不稳定——因为排序时，25*到了25的前面。

简单选择排序的算法如下：（亦可参见教材P277）

```
Void SelectSort(SqList  &L ) {        //对顺序表L作简单选择排序
   for (i=1;  i<L.length; ++i){
    j = SelectMinKey(L,i);   //在r[i…L.length]中选择最小记录并定位
   if( i!=j )  r[i] ↔r[j];              //与第i个记录交换
    } //for
 }  //SelectSort
```

讨论：能否利用（或记忆）首趟的n-1次比较所得信息，从而尽量减少后续比较次数呢？

答：能！请看——

锦标赛排序和堆排序

2） 锦标赛排序（又称树形选择排序）

基本思想：与体育比赛时的淘汰赛类似。

首先对 n 个记录的关键字进行两两比较，得到$\lceil n/2 \rceil$个优胜者(关键字小者)，作为第一步比较的结果保留下来。然后在这$\lceil n/2 \rceil$个较小者之间再进行两两比较，…，如此重复，直到选出最小关键字的记录为止。

优点：减少比较次数，加快排序速度

缺点：空间效率低

例：关键字序列T=（21，25，49，25*，16，08，63），请给出锦标赛排序的具体实现过程。

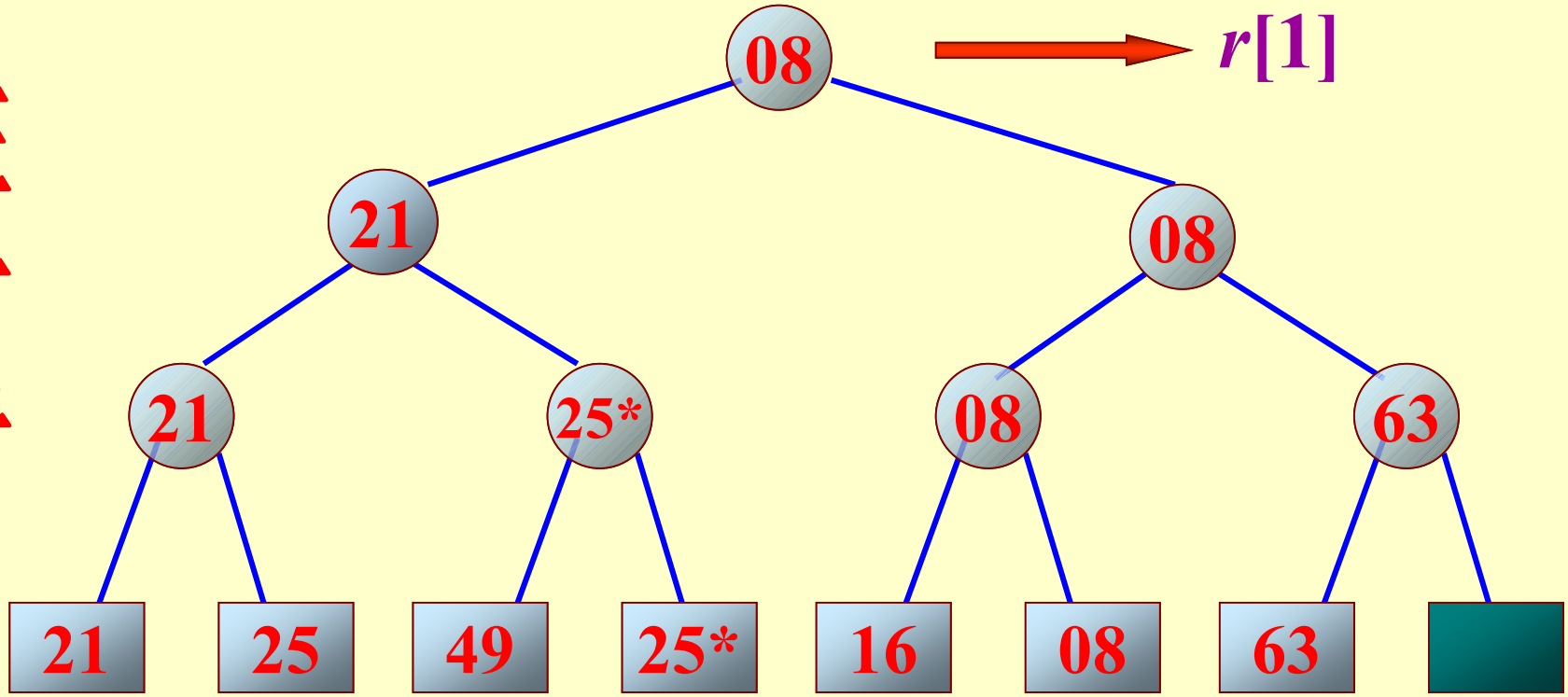
注：为便于自动处理，建议每个记录多开两个特殊分量：

key	otherinfo	Index(结点位置编号)	Tag（是否参加比较）

第一趟：

Winner (胜者)

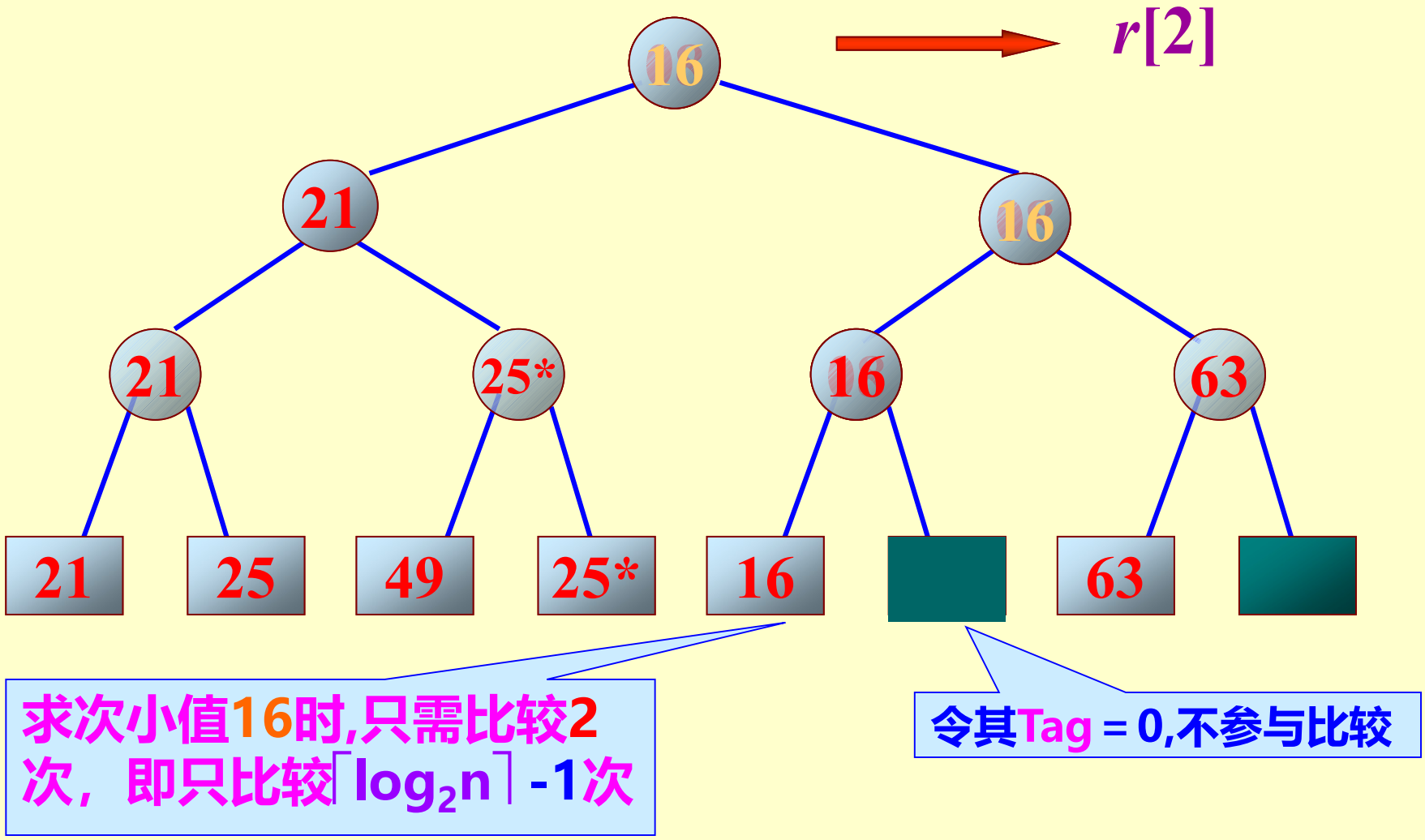
胜者树



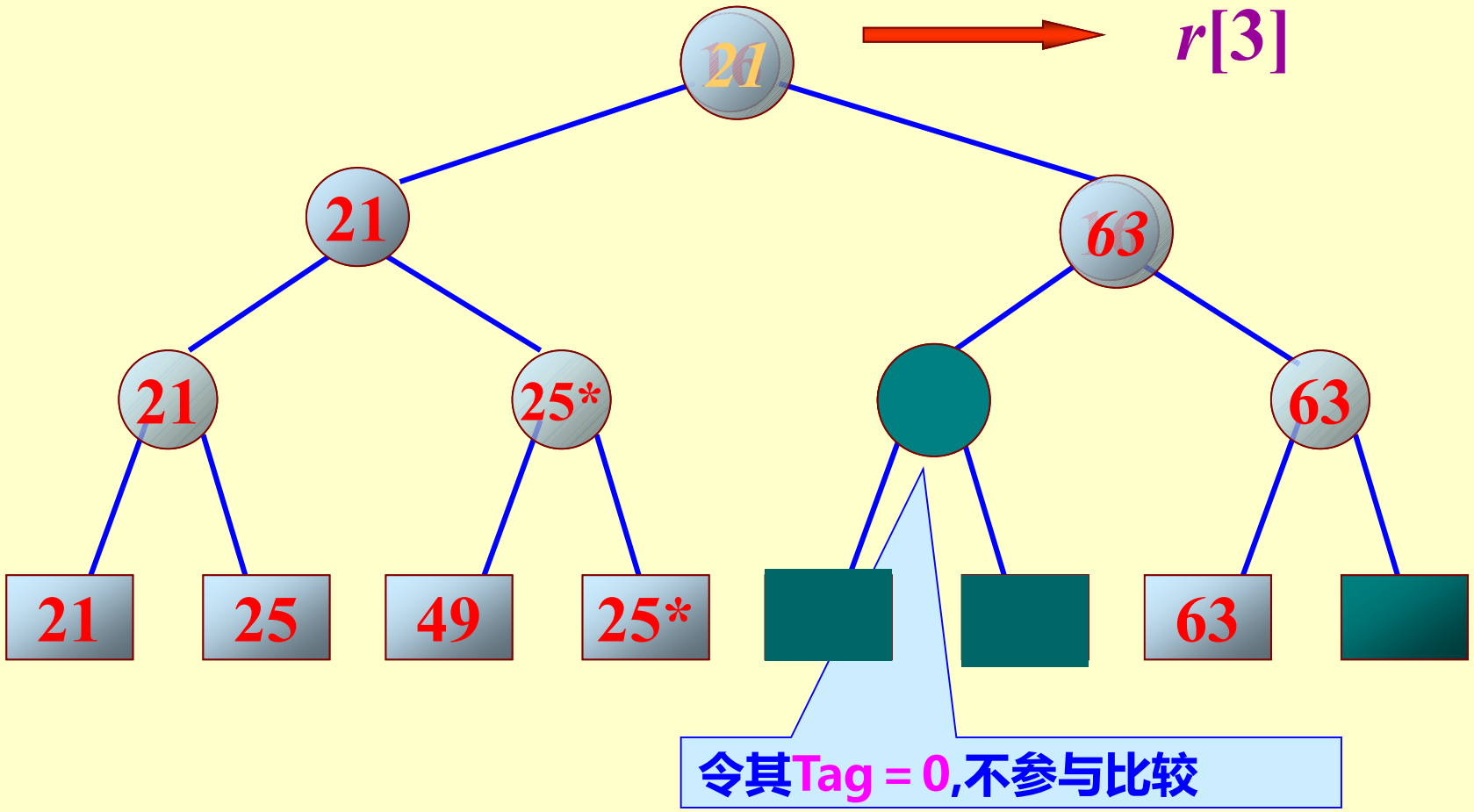
初态：

补足2^k($k=\lceil \log_2 n \rceil$)个叶子结点

第二趟：　　*Winner* (胜者)

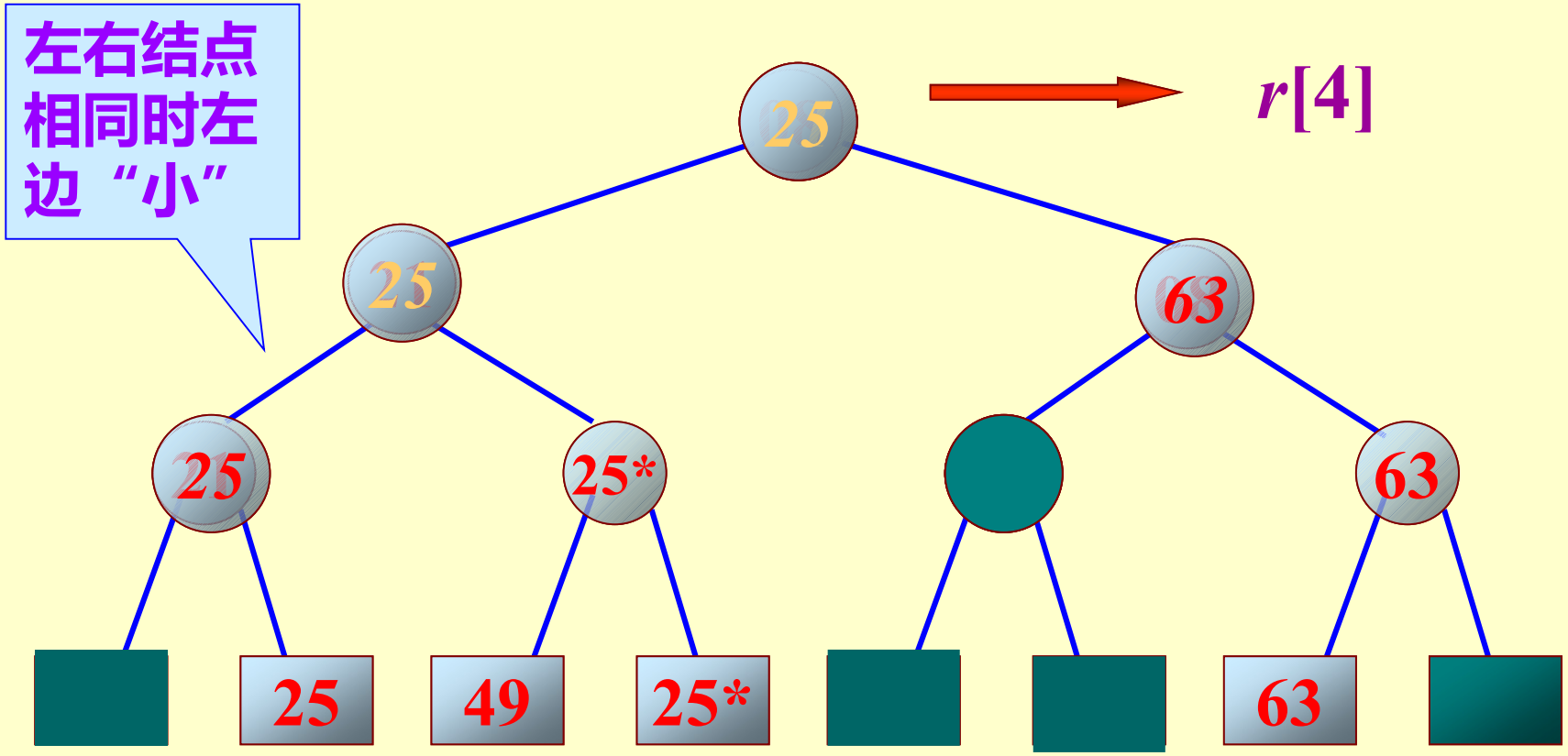


第三趟：　　*Winner* (胜者)



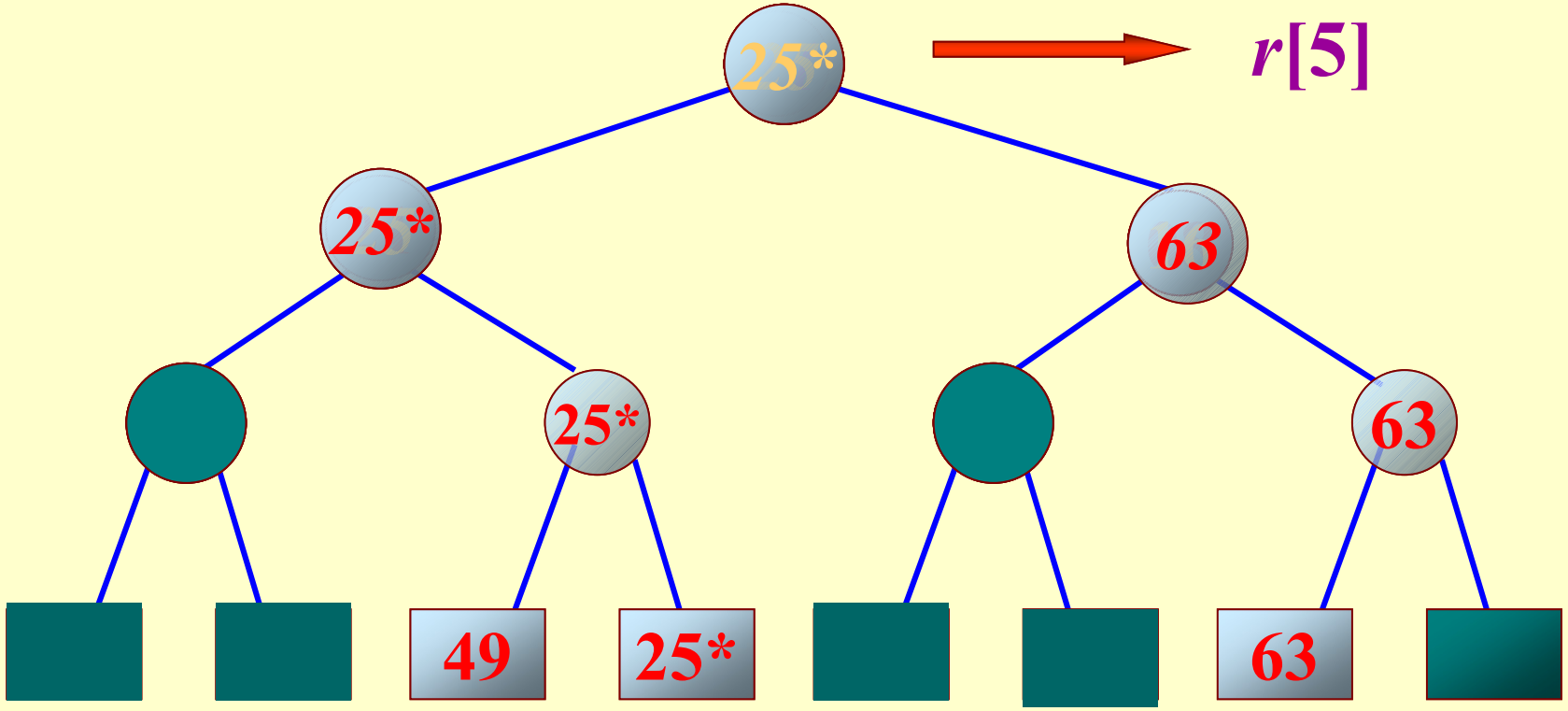
第四趟：

Winner (胜者)



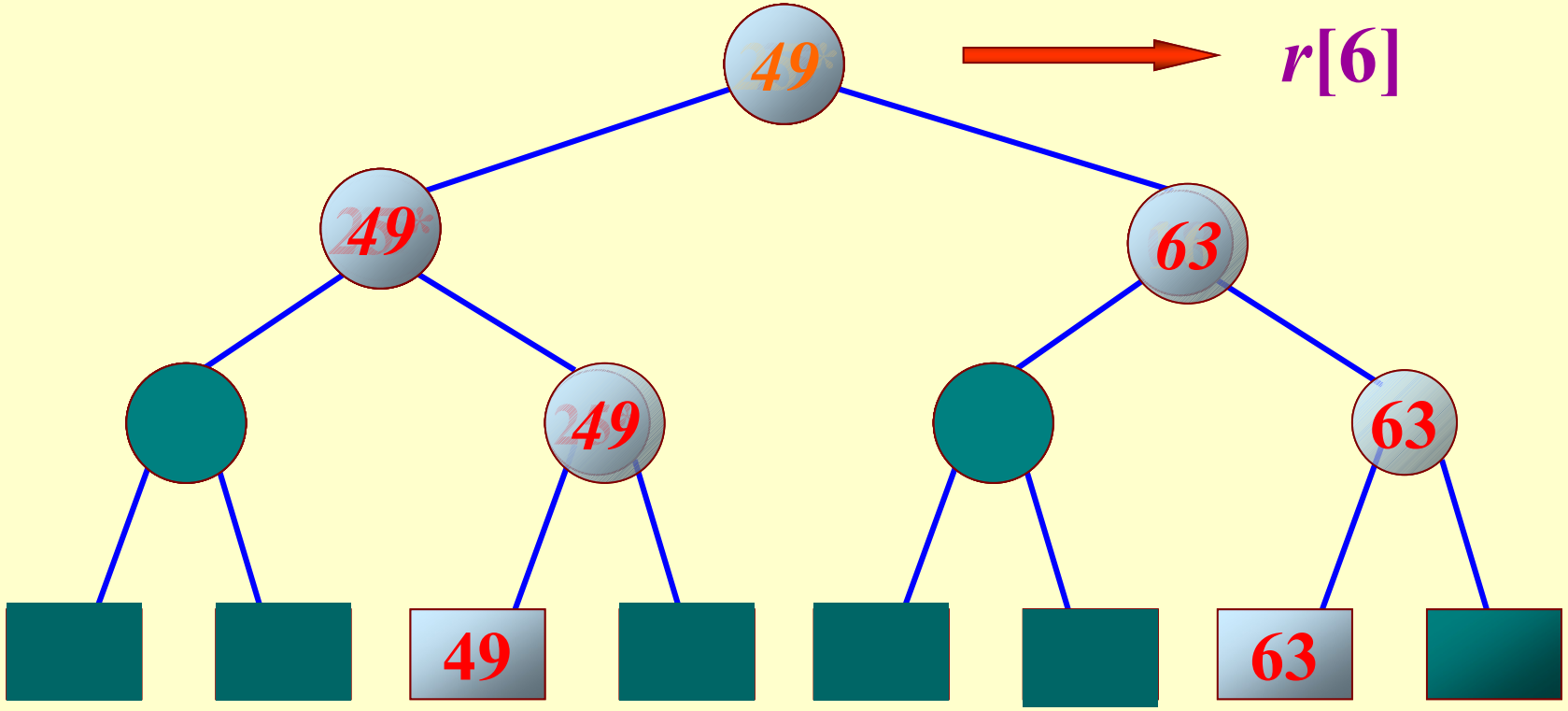
第五趟：

Winner (胜者)

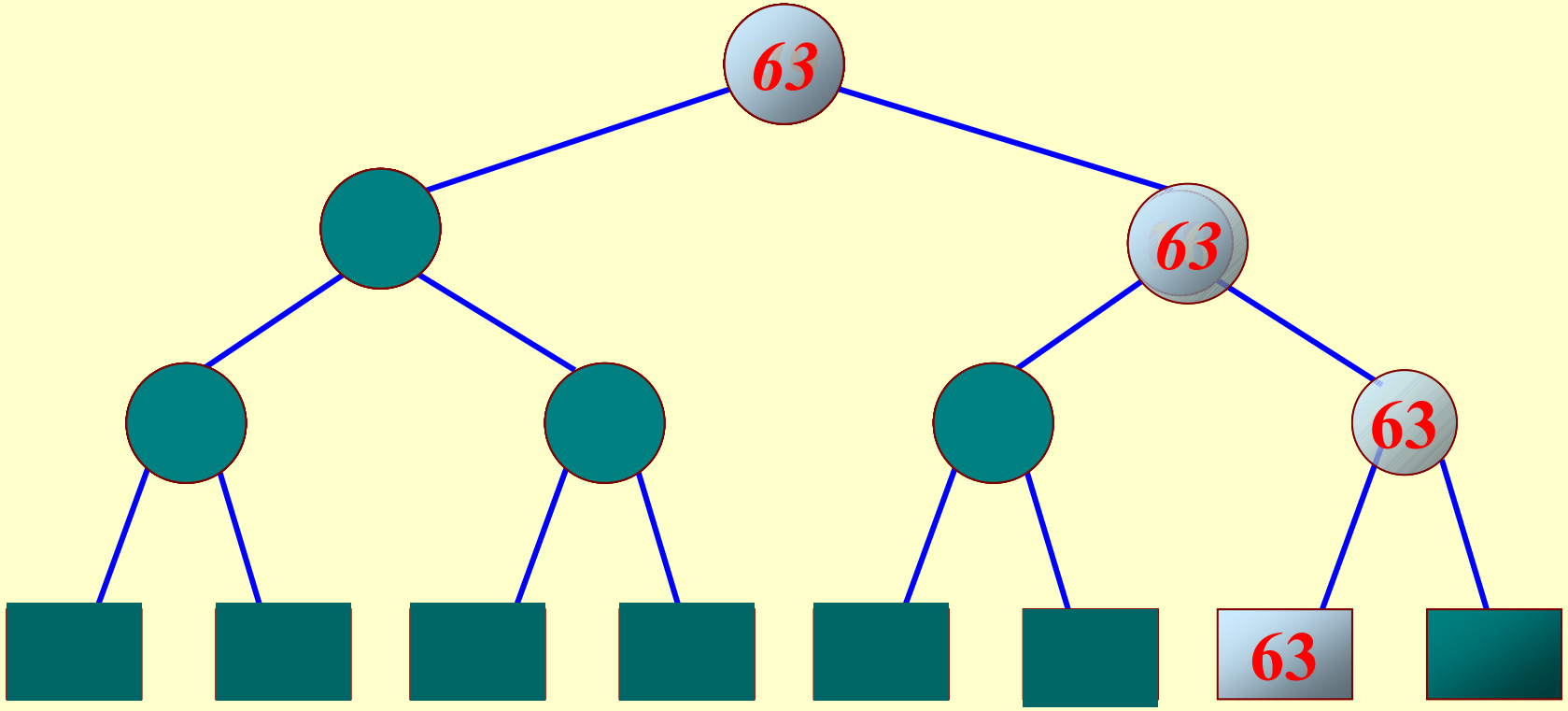


第六趟：

Winner (胜者)



第七趟: *Winner* (胜者) → *r*[7]



算法分析：

- 锦标赛排序构成的树是完全(满) 二叉树，其深度为 $\lfloor \log_2 n \rfloor + 1$，其中 n 为待排序元素个数。
- 时间复杂度：$O(n\log_2 n)$ —n个记录各自比较约$\log_2 n$次
- 空间效率：$O(n)$ —胜者树的附加内结点共有n_0-1个！
- 稳定性：稳定 —可事先约定左结点“小”

$n_0 = 2^k$ = 叶子总数

讨论：锦标赛排序巧妙利用了第一次和前若干次扫描的结果，加快了排序速度。还有没有同类方法？

答：有！请看——

堆排序

3） 堆排序

1. 什么是堆?　　2. 怎样建堆?　　3. 怎样堆排序?

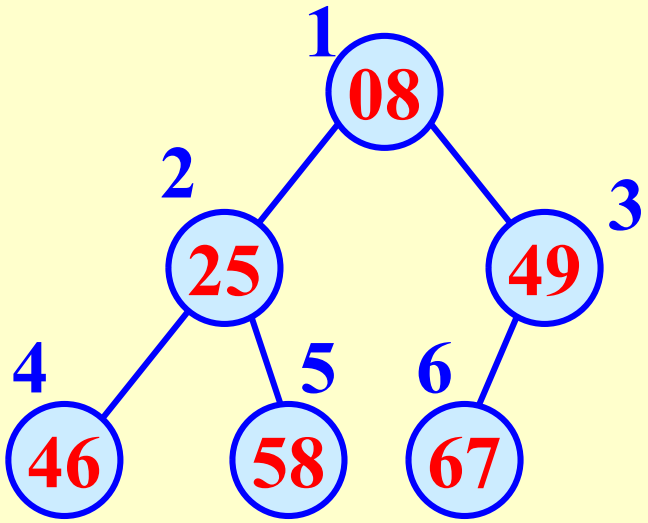
堆的定义：设有n个元素的序列 k_1，k_2，…，k_n，当且仅当满足下述关系之一时，称之为堆。

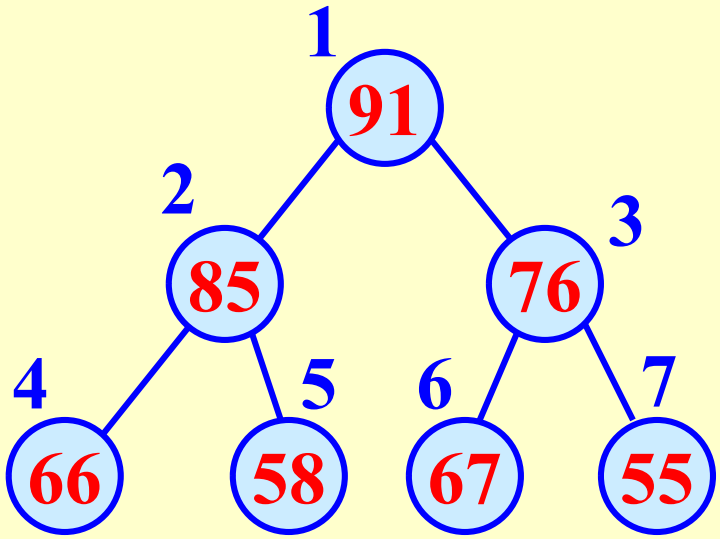
$$\begin{cases} k_i \le k_{2i} \\ k_i \le k_{2i+1} \end{cases} \quad \text{或者} \quad \begin{cases} k_i \ge k_{2i} \\ k_i \ge k_{2i+1} \end{cases} \qquad i=1, 2,\ldots n/2$$

解释：如果让满足以上条件的元素序列（k_1，k_2，…，k_n）顺次排成一棵完全二叉树，则此树的特点是：

树中所有结点的值均大于（或小于）其左右孩子，此树的根结点（即堆顶）必最大（或最小）。

例： 有序列T1=（08, 25, 49, 46, 58, 67）和序列T2=（91, 85, 76, 66, 58, 67, 55）,判断它们是否“堆”？





√ （小根堆）
（小顶堆）
（最小堆）

√ （大根堆）
（大顶堆）
（最大堆）

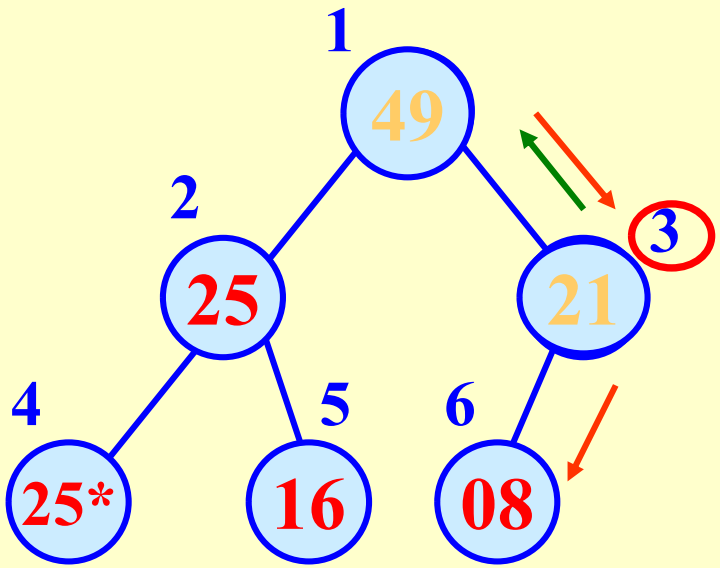
2. 怎样建堆?

步骤：从最后一个非终端结点开始往前逐步调整，让每个双亲大于（或小于）子女，直到根结点为止。

终端结点（即叶子）没有任何子女，无需单独调整

例：关键字序列T= (21，25，49，25*，16，08），请建大根堆。

解：为便于理解，先将原始序列画成完全二叉树的形式：这样可以很清晰地从⌊n/2⌋开始调整。



完全二叉树的第一个非终端结点编号必为⌊n/2⌋！！(性质5)

i=3: 49大于08，不必调整;

*i=2: 25大于25*和16，也不必调整;*

i=1: 21小于25和49，要调整!

而且21还应当向下比较！！

建堆算法 (等同于堆排序算法中的第一步)

参见教材P281-282

```
void HeapSort (HeapType &H ) {
                    //H是顺序表，含有H.r[ ]和H.length两个分量
  for ( i = length / 2;  i >0; - - i ) //把r[1…length]建成大根堆
    HeapAdjust(r, i, length );    //使r[i…length]成为大根堆
  ……
} // HeapSort
```

*HeapAdjust*是针对结点 *i* 的堆调整函数，其含义是：**从结点i开始到堆尾为止，自上向下比较，如果子女的值大于双亲结点的值，则互相交换，即把局部调整为大根堆。**

针对结点 *i* 的堆调整函数*HeapAdjust* 表述如下：

——从结点i开始到当前堆尾*m*为止，自上向下比较，如果子女的值大于双亲结点的值，则互相交换，即把局部调整为大根堆。

```
HeapAdjust(r, i, m ){
current=i; temp=r[i]; child=2*i;   //temp暂存r[i]值，child是其左孩子
 while(child<=m){        //检查是否到达当前堆尾，未到尾则整理
  if ( child<m && r[child].key<r[child+1].key )
  child= child+1;       //让child指向两子女中的大者位置
  if ( temp.key>=r[child].key ) breack; //根大则不必调整，函数结束
  else {   r[current]=r[child];      //否则子女中的大者上移
  current= child; child=2* child; } //将根下降到子女位置并继续向下整理!
        }// while
    r[current]=temp;  //直到自下而上都满足堆定义，再安置入口结点
  }  // HeapAdjust
```

3. 怎样进行整个序列的堆排序?

难点： 将堆的当前顶点输出后，如何将剩余序列重新调整为堆?

方法： 将当前顶点与堆尾记录交换，然后仿建堆动作重新调整，如此反复直至排序结束。

即：将任务转化为—>

H.r[i…m]中除r[i]外，其他都具有堆特征。

现调整r[i]的值，使H.r[i…m]为堆。

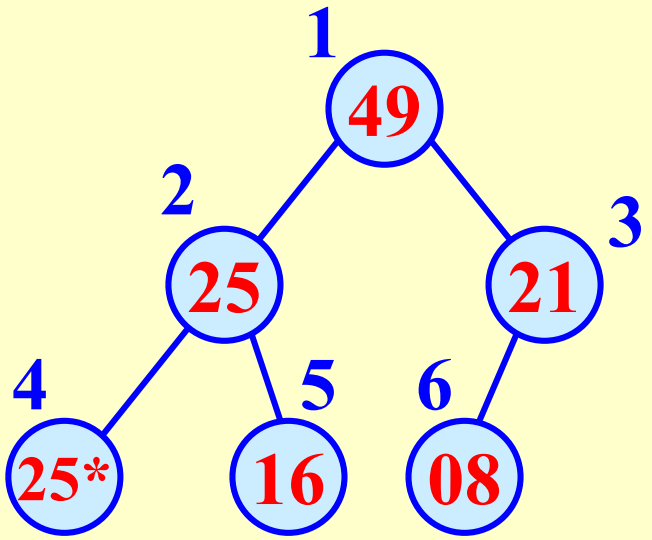
基于初始堆进行堆排序的算法步骤：

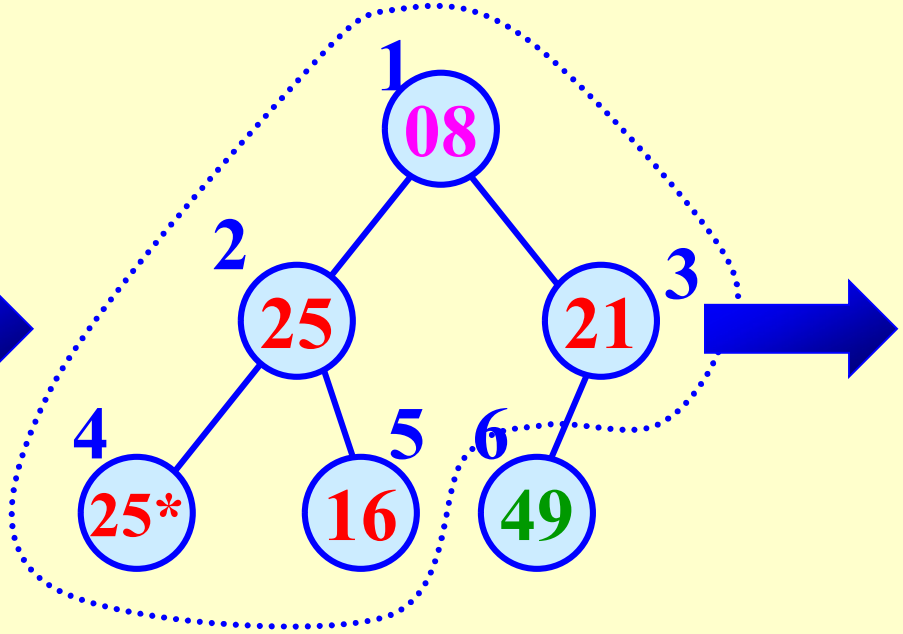
堆的第一个对象$r[0]$具有最大的关键码，将$r[0]$与$r[n]$对调，把具有最大关键码的对象交换到最后；

再对前面的n-1个对象，使用堆的调整算法，重新建立堆。结果具有次最大关键码的对象又上浮到堆顶，即$r[0]$位置；

再对调$r[0]$和$r[n-1]$，然后对前n-2个对象重新调整，…如此反复，最后得到全部排序好的对象序列。

例：对刚才建好的大根堆进行排序：





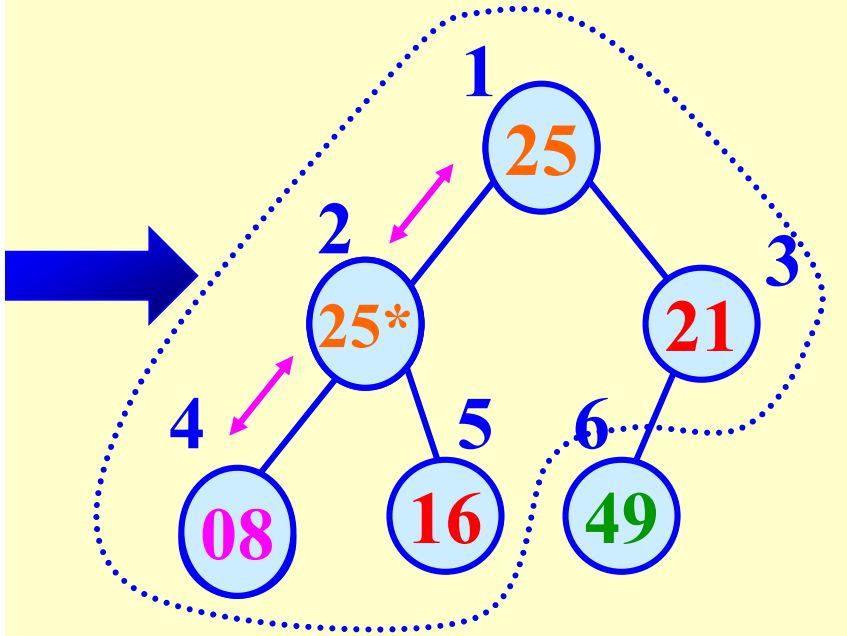
49	25	21	25*	16	08

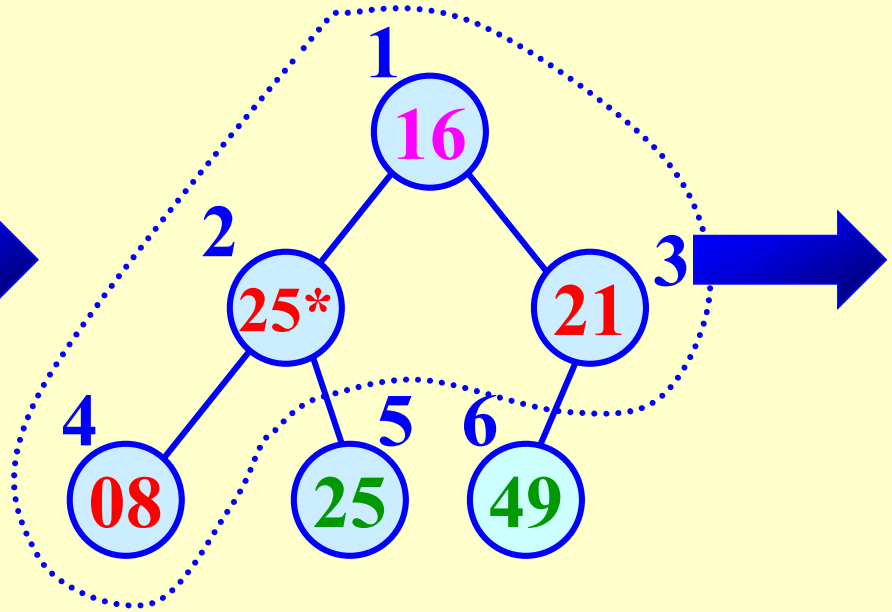
初始最大堆

08	25	21	25*	16	49

交换 1号与 6 号记录

r[0]　　r[n]



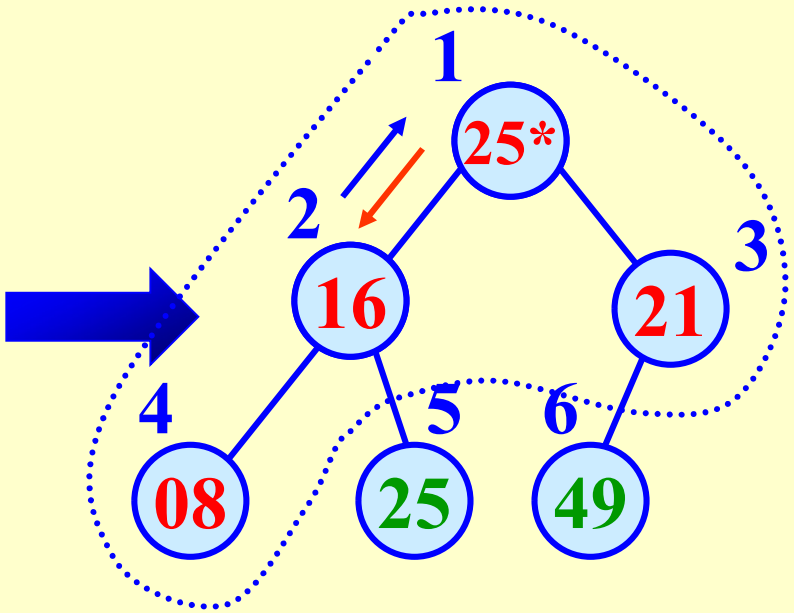


25	25*	21	08	16	49

从 1 号到 5 号 重新调整为最大堆

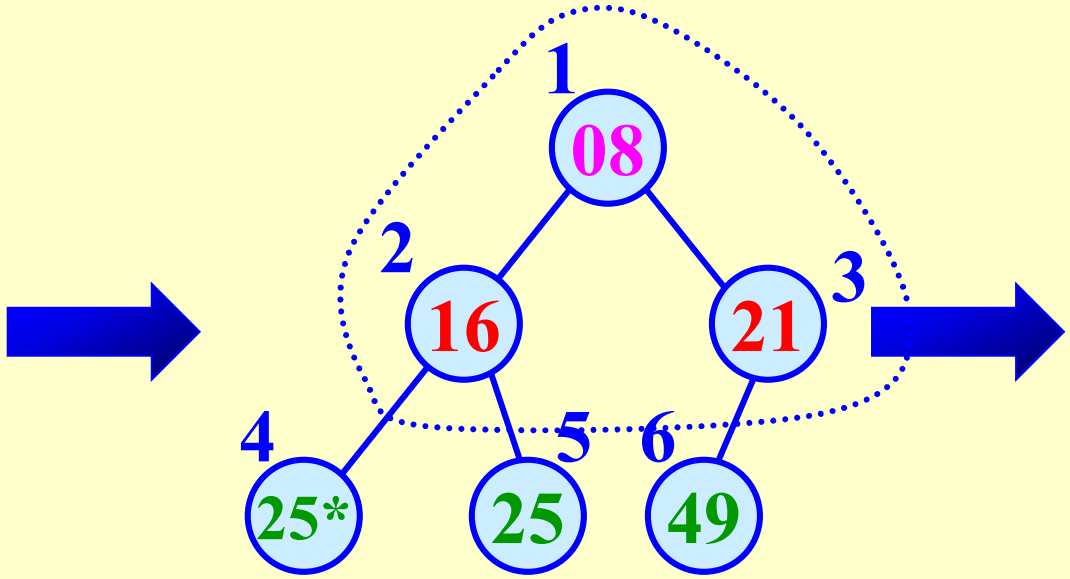
16	25*	21	08	25	49

交换 1号与 5 号记录



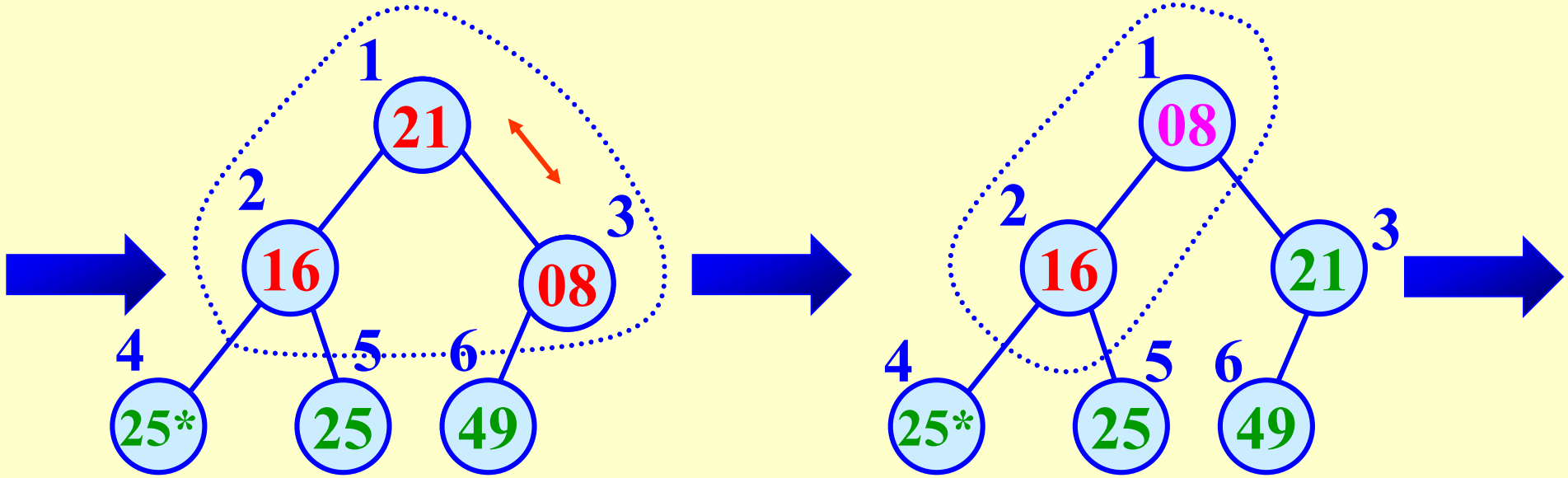
25*	16	21	08	25	49

从 1号到 4号 重新调整为最大堆



08	16	21	25*	25	49

交换 1 号与 4 号记录

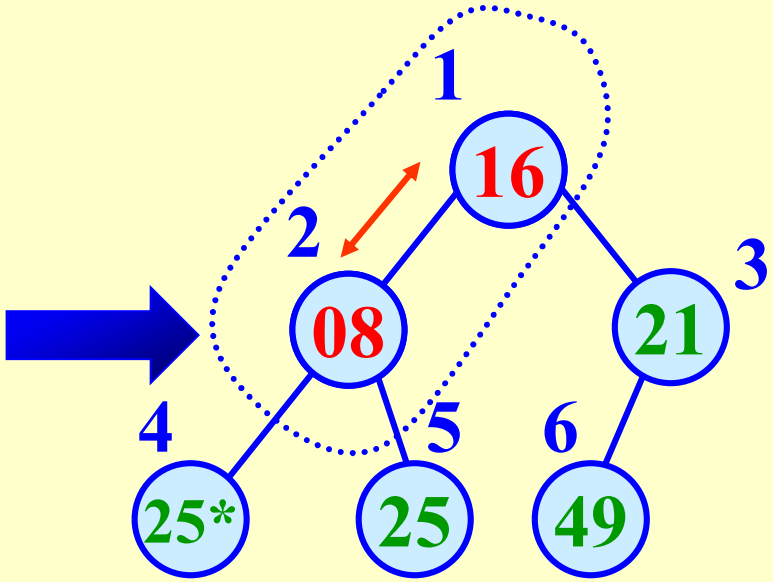


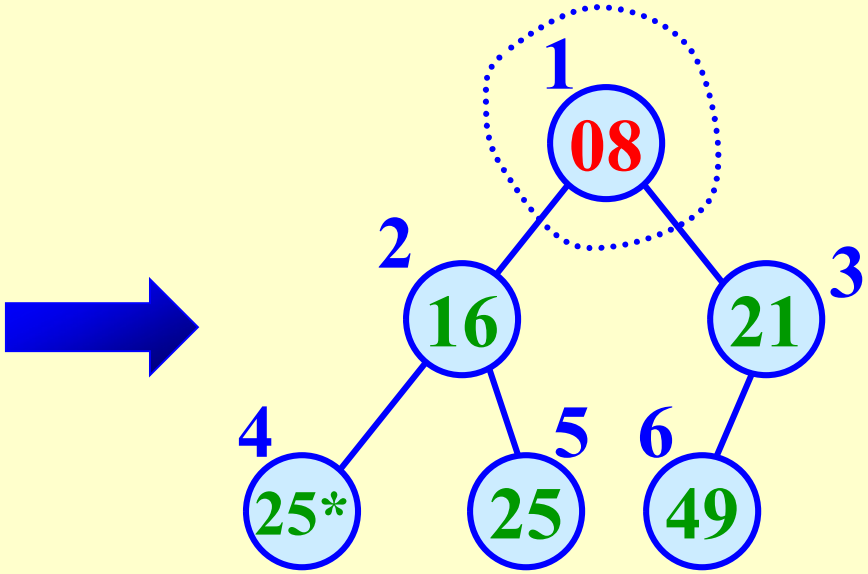
21	16	08	25*	25	49

从 1 号到 3号 重新调整为最大堆

08	16	21	25*	25	49

交换 1号与3 号记录





16	08	21	25*	25	49

从 1 号到 2 号 重新调整为最大堆

08	16	21	25*	25	49

交换 1 号与2 号记录排序完毕。

堆排序的算法　　参见教材P281-282

```
void HeapSort (HeapType &H ) {
//对顺序表H进行堆排序
   for ( i = H.length / 2;  i >0; - - i )
      HeapAdjust(H,i, H.length );     //for,建立初始堆
   for ( i = H.length; i > 1; - -i) {
      H.r[1] ←→ H.r[i];      //交换，要借用temp
      HeapAdjust( H, 1,i-1 );  //重建最大堆，i-1=m
   }
}
```

因为每次从堆顶开始调整，故每次调用耗时 $O(\log_2 n)$

堆排序算法分析：

- 时间效率： $O(n\log_2 n)$。因为整个排序过程中需要调用n-1次*HeapAdjust*( )算法，而算法本身耗时为$\log_2 n$；
- 空间效率：O(1)。仅在第二个for循环中交换记录时用到一个临时变量temp。
- 稳定性： 不稳定。
- 优点：对小文件效果不明显，但对大文件有效。